



AI Driven SQA: Manual & Automation Testing

Assignment 05

Module: 05

Topic: Problem Solving with AI (Claude Code + JavaScript)

Submitted by

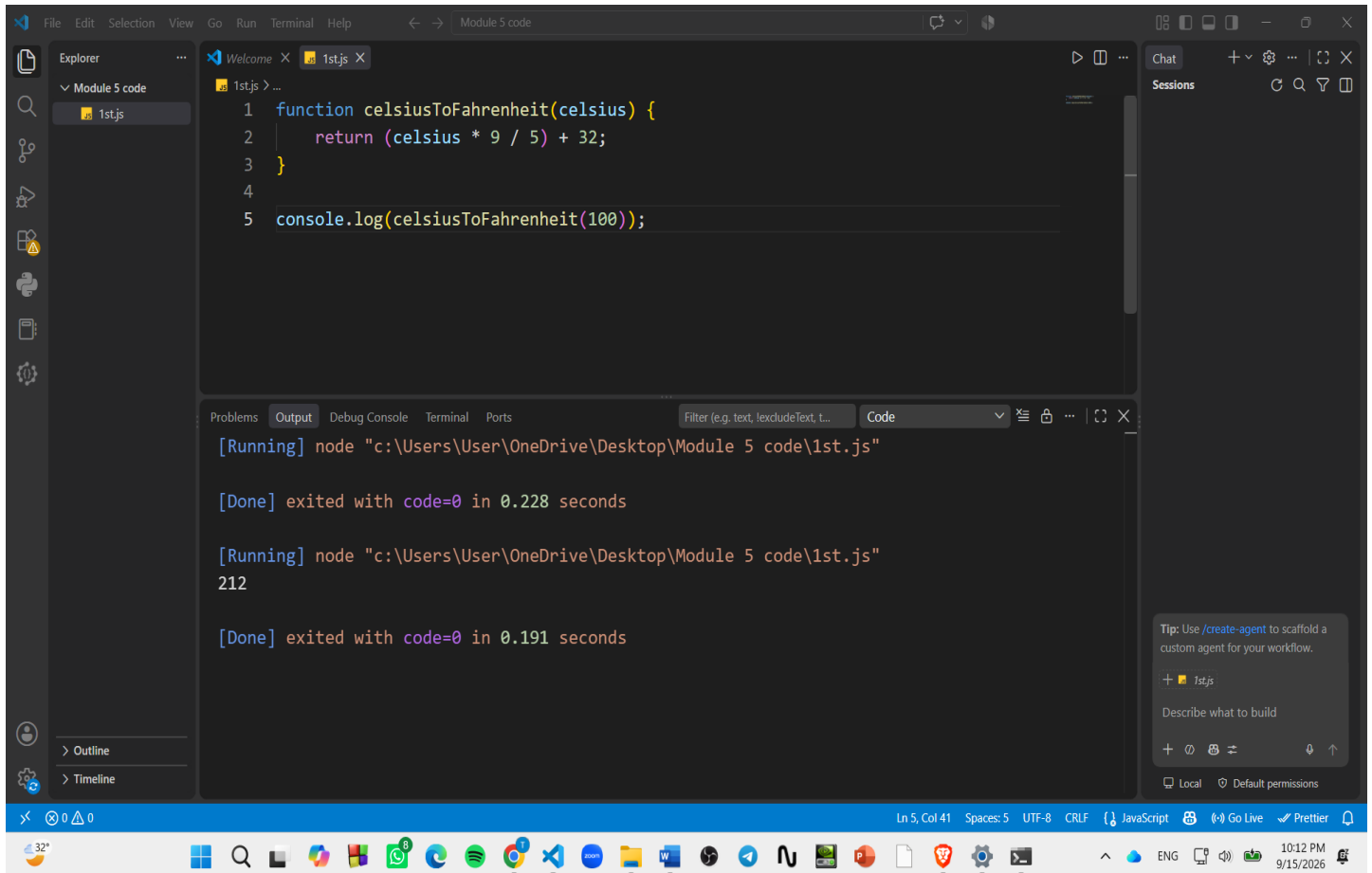
Name: Tahsinur Rahman Tanjim

Batch: 21

Phone: 01790576143

Q1. JavaScript Coding Practice

1. Convert Celsius to Fahrenheit



The screenshot shows the Visual Studio Code editor with a file named `1st.js` open. The code defines a function `celsiusToFahrenheit` that takes a parameter `celsius` and returns the equivalent Fahrenheit temperature using the formula $(celsius * 9 / 5) + 32$. The function is called with `100` and the result is logged to the console.

```
1 function celsiusToFahrenheit(celsius) {  
2   return (celsius * 9 / 5) + 32;  
3 }  
4  
5 console.log(celsiusToFahrenheit(100));
```

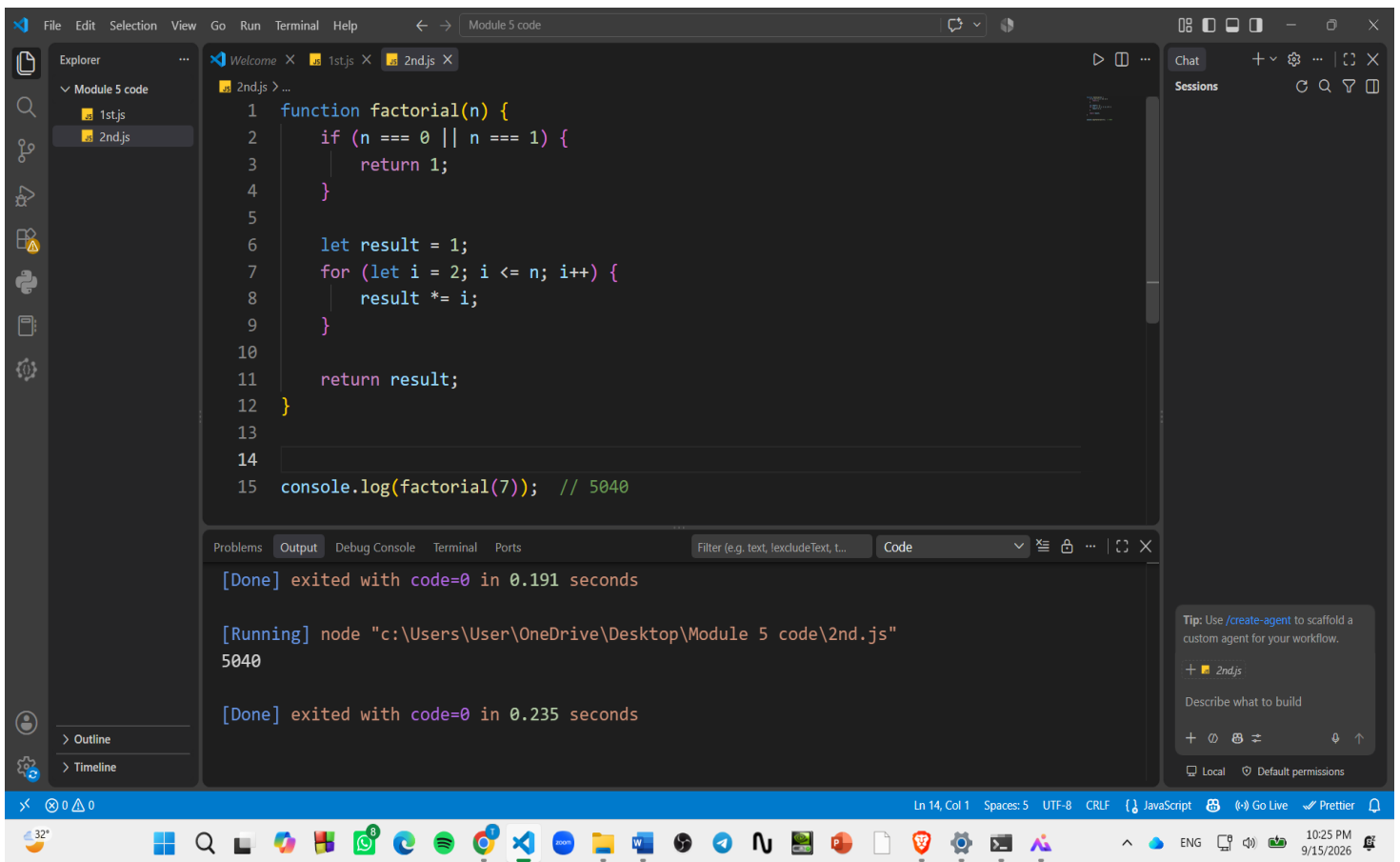
The Output window shows the execution results:

```
[Running] node "c:\Users\User\OneDrive\Desktop\Module 5 code\1st.js"  
  
[Done] exited with code=0 in 0.228 seconds  
  
[Running] node "c:\Users\User\OneDrive\Desktop\Module 5 code\1st.js"  
212  
  
[Done] exited with code=0 in 0.191 seconds
```

Explanation:

1. The function takes one parameter, `celsius`, representing the temperature in Celsius.
2. It applies the formula $(celsius * 9/5) + 32$ and returns the result the equivalent temperature in Fahrenheit.
3. You can call it with any number and log the result to see the conversion.

2. Find the Factorial of a Number



The screenshot shows a Visual Studio Code editor window with a file named '2nd.js' open. The code defines a 'factorial' function that uses a for loop to calculate the factorial of a given number 'n'. The function returns 1 for base cases (0 or 1) and multiplies the result by each integer from 2 to 'n'. A console log statement at the bottom calls 'factorial(7)', which outputs '5040'. The bottom panel shows the 'Output' window with the execution log, confirming the result of 5040.

```
1 function factorial(n) {  
2   if (n === 0 || n === 1) {  
3     return 1;  
4   }  
5  
6   let result = 1;  
7   for (let i = 2; i <= n; i++) {  
8     result *= i;  
9   }  
10  
11   return result;  
12 }  
13  
14  
15 console.log(factorial(7)); // 5040
```

[Done] exited with code=0 in 0.191 seconds

[Running] node "c:\Users\User\OneDrive\Desktop\Module 5 code\2nd.js"

5040

[Done] exited with code=0 in 0.235 seconds

Explanation:

1. function factorial(n) takes a number n and will return its factorial ($n! = n \times (n-1) \times \dots \times 1$).
2. Base case: if n is 0 or 1, return 1 right away (by definition).
3. let result = 1; starts an accumulator at 1.
4. The loop starts at i = 2 and runs while i <= n, incrementing i each time.
5. Each pass: result *= i multiplies result by i.
6. For factorial(5): result goes $1 \rightarrow 2 \rightarrow 6 \rightarrow 24 \rightarrow 120$.
7. Loop stops once i exceeds n.

8. return result; gives back the final product.
9. Uses iteration (loop) instead of recursion more memory efficient for large n.

3. Check for Palindrome

The screenshot shows a Visual Studio Code editor with a file explorer on the left containing '1st.js', '2nd.js', and '3rd.js'. The main editor window displays the following JavaScript code in '3rd.js':

```
1 function isPalindrome(word) {  
2   const reversed = word.split("").reverse().join("");  
3   return word === reversed;  
4 }  
5  
6  
7 console.log(isPalindrome("hello"));  
8
```

Below the code editor, the Output window shows the execution results:

```
[Done] exited with code=0 in 0.235 seconds  
  
[Running] node "c:\Users\User\OneDrive\Desktop\Module 5 code\3rd.js"  
false  
  
[Done] exited with code=0 in 0.269 seconds
```

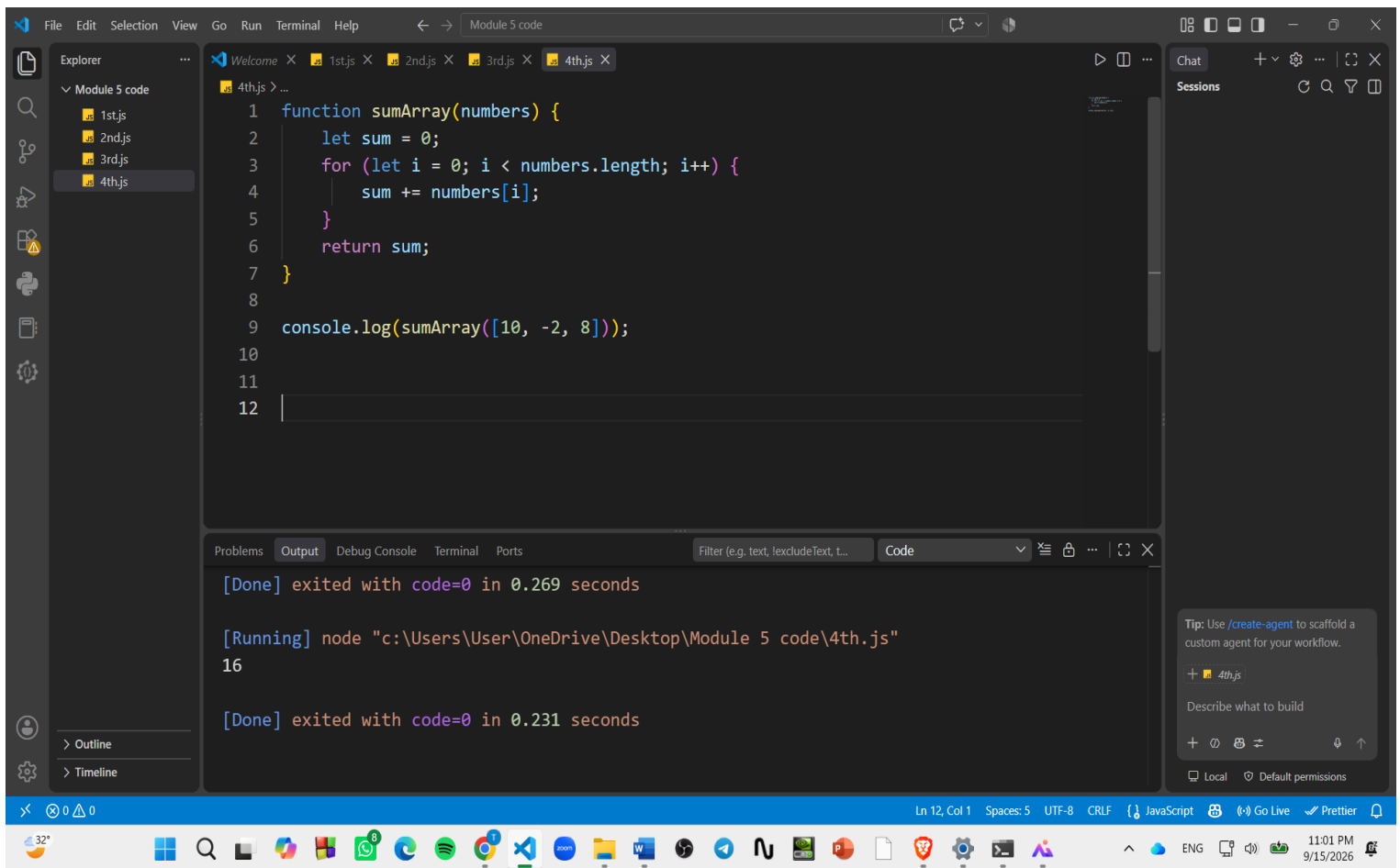
The status bar at the bottom indicates the file is 'Ln 8, Col 1' with 'Spaces: 5' and 'UTF-8' encoding. The system tray at the bottom shows the date and time as '10:53 PM 9/15/2026'.

Explanation:

1. function isPalindrome(word) takes a string word to check.
2. word.split("") breaks the string into an array of individual characters.

3. `.reverse()` reverses the order of that array.
4. `.join("")` combines the reversed array back into a string.
5. `word === reversed` compares the original string to the reversed .
6. Returns true if they match (it's a palindrome), false otherwise.

4. Sum of Array Elements



The screenshot shows the Visual Studio Code editor with a project named 'Module 5 code'. The Explorer sidebar on the left shows four files: 1st.js, 2nd.js, 3rd.js, and 4th.js. The main editor window displays the code in 4th.js:

```
1 function sumArray(numbers) {  
2   let sum = 0;  
3   for (let i = 0; i < numbers.length; i++) {  
4     sum += numbers[i];  
5   }  
6   return sum;  
7 }  
8  
9 console.log(sumArray([10, -2, 8]));  
10  
11  
12
```

Below the code editor, the Output window shows the execution results:

```
[Done] exited with code=0 in 0.269 seconds  
  
[Running] node "c:\Users\User\OneDrive\Desktop\Module 5 code\4th.js"  
16  
  
[Done] exited with code=0 in 0.231 seconds
```

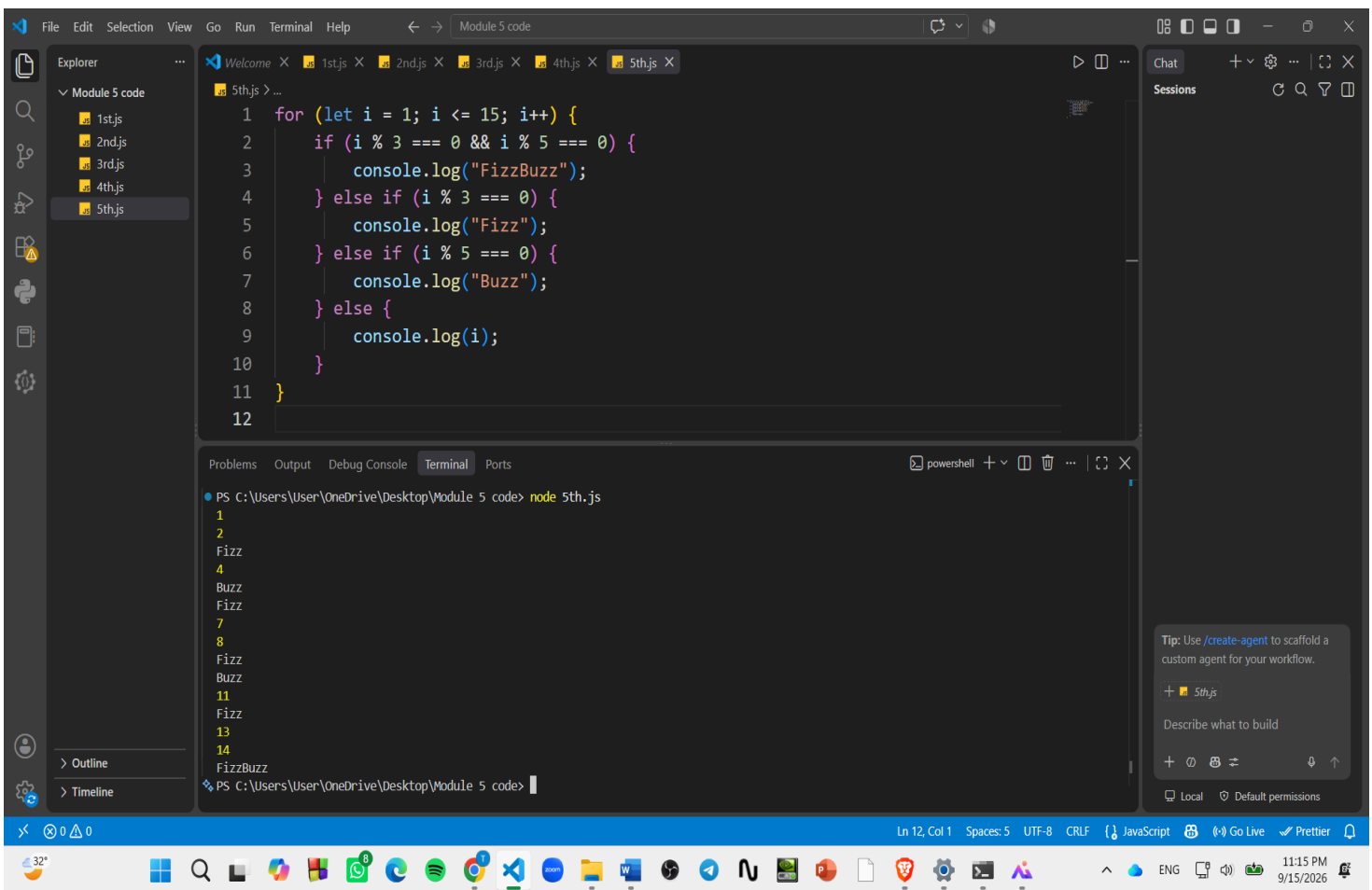
The status bar at the bottom indicates the current position is Line 12, Column 1, with 5 spaces, UTF-8 encoding, and CRLF line endings. The language is set to JavaScript, and the Prettier formatter is active.

Explanation:

1. `function sumArray(numbers)` takes an array of numbers.
2. `let sum = 0;` starts an accumulator at 0.

3. The loop runs through each index of the array, from 0 to `numbers.length - 1`.
4. Each pass: `sum += numbers[i]` adds the current element to `sum`.
5. `return sum`; gives back the total after the loop finishes.

5. FizzBuzz (Basic)



The screenshot shows a Visual Studio Code editor with a file explorer on the left containing files 1st.js through 5th.js. The main editor window displays the code for 5th.js, which implements a FizzBuzz algorithm. The code is as follows:

```
1 for (let i = 1; i <= 15; i++) {  
2   if (i % 3 === 0 && i % 5 === 0) {  
3     console.log("FizzBuzz");  
4   } else if (i % 3 === 0) {  
5     console.log("Fizz");  
6   } else if (i % 5 === 0) {  
7     console.log("Buzz");  
8   } else {  
9     console.log(i);  
10  }  
11 }  
12
```

Below the code editor is a terminal window with the command `node 5th.js` executed. The output of the program is:

```
1  
2  
Fizz  
4  
Buzz  
Fizz  
7  
8  
Fizz  
Buzz  
11  
Fizz  
13  
14  
FizzBuzz
```

The status bar at the bottom indicates the current line is 12, column 1, with 5 spaces, UTF-8 encoding, and CRLF line endings. The language is set to JavaScript, and the editor has Prettier formatting enabled. The system tray at the very bottom shows the date and time as 11:15 PM on 9/15/2026.

Explanation:

1. The loop runs `i` from 1 to 15.

2. First check: $i \% 3 === 0 \ \&\& \ i \% 5 === 0$ if i is divisible by both 3 and 5, print "FizzBuzz".
3. Else if $i \% 3 === 0$ divisible by 3 only, print "Fizz".
4. Else if $i \% 5 === 0$ divisible by 5 only, print "Buzz".
5. Otherwise, print the number i itself.